

Process MeNtOR 3.0

Uni-SEP

Legends of Sword and Wand Design Document

Version:	2.0
Print Date:	01/31/2026
Release Date:	03/25/2026
Release State:	Draft
Approval State:	Pending
Approved by:	N/A
Prepared by:	Ethan Persaud, Mohammed Patel, Selimalp Bayrak, William Bui
Reviewed by:	N/A
Path Name:	/Deliverable 2 & 3/LegendsOfSwordAndWand_Design nDocument.pdf
File Name:	LegendsOfSwordAndWand_Design Document.pdf
Document No:	LSW-DD-01

Document Change Control

Version	Date	Authors	Summary of Changes
1.0	02/03/2026	Entire Team	Initial Design Document
2.0	03/23/2026	Entire Team	Updated Design Document (Deliverable 2)

Document Sign-Off

Name (Position)	Signature	Date
Mohammed Patel -221648423 (Developer)	Mohammed Patel	02/27/2026
Selimalp Bayrak - 221840814 (Developer)	Selimalp Bayrak	02/27/2026
Ethan Persaud - 221444567 (Developer)	Ethan Persaud	02/27/2026
William Bui - 221294475 (Developer)	William Bui	02/27/2026

Contents

1. Introduction	3
1.1. Purpose	4
1.2. Overview	4
1.3. Resources - References	4
2. Sequence Diagrams	5
3. Major Design Decisions	10
3.1. Layered Architecture	10
3.2. Domain-Driven Modularization	11
3.3. Use of Interfaces	11
3.4. Strategy-Based Battle Execution	11
3.5. Database Abstraction with Spring Data JPA	11
3.6. Prevention of God Classes	12
4. Architecture	12
4.1. Component Diagram	12
4.2.	12
4.3. Interface Specification	13
4.4. Architectural Patterns Used	19
4.5. Database Configuration	20
5. Detailed Class Diagrams	20
5.1. UML Class Diagrams	20
6. Use of Design Patterns	22
6.1. Strategy Pattern	22
6.2. Factory Pattern	23
6.3. Template Method Pattern	24
6.4. Decorator Pattern	25
6.5. Facade Pattern	25
6.6. Builder Pattern	26
7. Activities Plan	27
7.1. Gantt Chart	27
7.2. Project Backlog and Sprint Backlog	27
7.3. Group Meeting Logs	29
8. Test Driven Development	30

1. Introduction

1.1. Purpose

This document details the architectural and design specifications of the *Legends of Sword and Wand* system.

It describes the structural decomposition, component interactions, class design and design patterns employed in the implementation of the system.

The purpose of this document is to serve as a technical reference for developers, testers and stakeholders involved in the development, validation and maintenance of the system. It ensures architectural consistency, traceability to use cases and alignment with project requirements.

GitHub Repository:

<https://github.com/MohammedPatel-yorku/LegendsOfSwordAndWand>

1.2. Overview

Legends of Sword and Wand is a modular tactical role-playing game system implemented using Spring Boot and MySQL. The system supports:

- User profile creation and authentication
- Player-versus-Environment campaign progression
- Turn-based battle mechanics
- Persistent storage of party, hero and campaign state

The system is designed using a layered architecture that separates presentation logic, business logic and data persistence. This design promotes scalability, maintainability and adherence to high cohesion and low coupling principles.

This design document is organized as follows:

- **Cover:** Main Page and document revision history
- **Section 1:** Introduction and project context
- **Section 2:** Major architectural and modularization decisions
- **Section 3:** Sequence diagrams for all primary use cases
- **Section 4:** System architecture and module decomposition
- **Section 5:** UML class diagrams for implemented components
- **Section 6:** Design patterns and justification
- **Section 7:** Project planning, backlog and Gantt chart
- **Section 8:** Test-Driven Development strategy and test cases

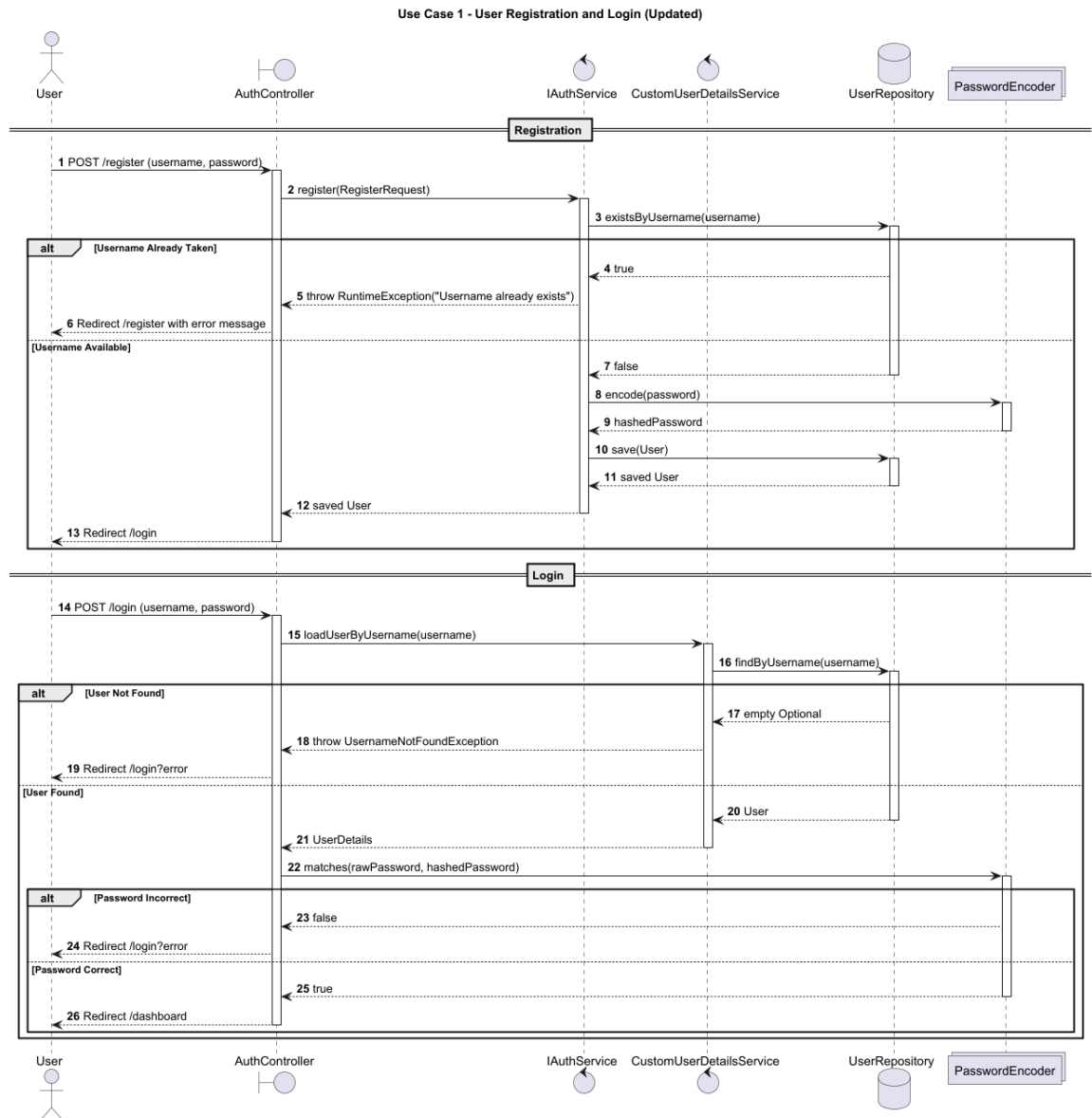
1.3. Resources - References

The following resources were used to aid the development of the system:

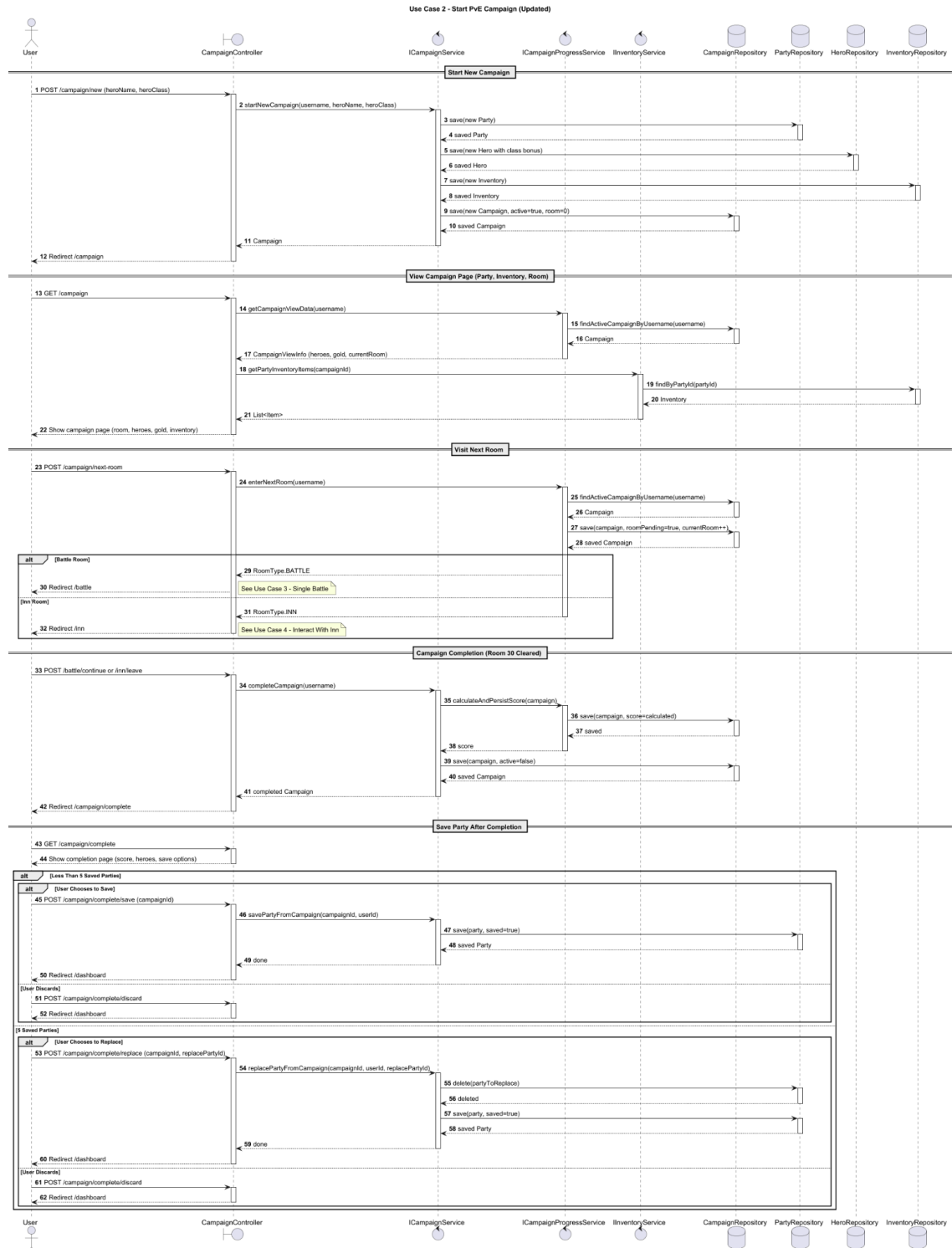
- “Project Description - Legends of Sword and Wand”
- “Deliverable 1: Instructions”
- “Project Requirements: Use Cases”
- Spring Boot Documentation
- Spring Data JPA Documentation
- UML 2.x Specification

2. Sequence Diagrams

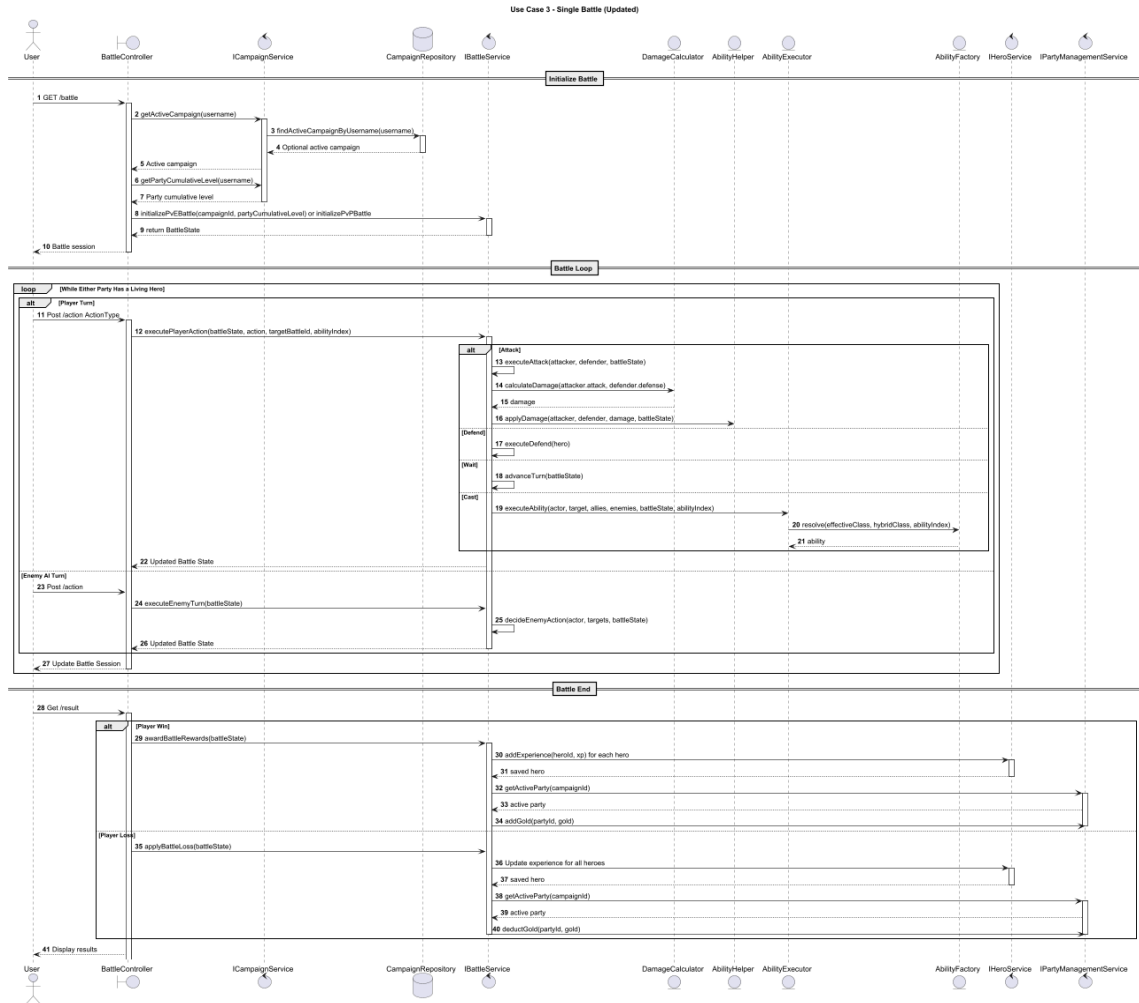
- Use Case 1: User Registration and Login



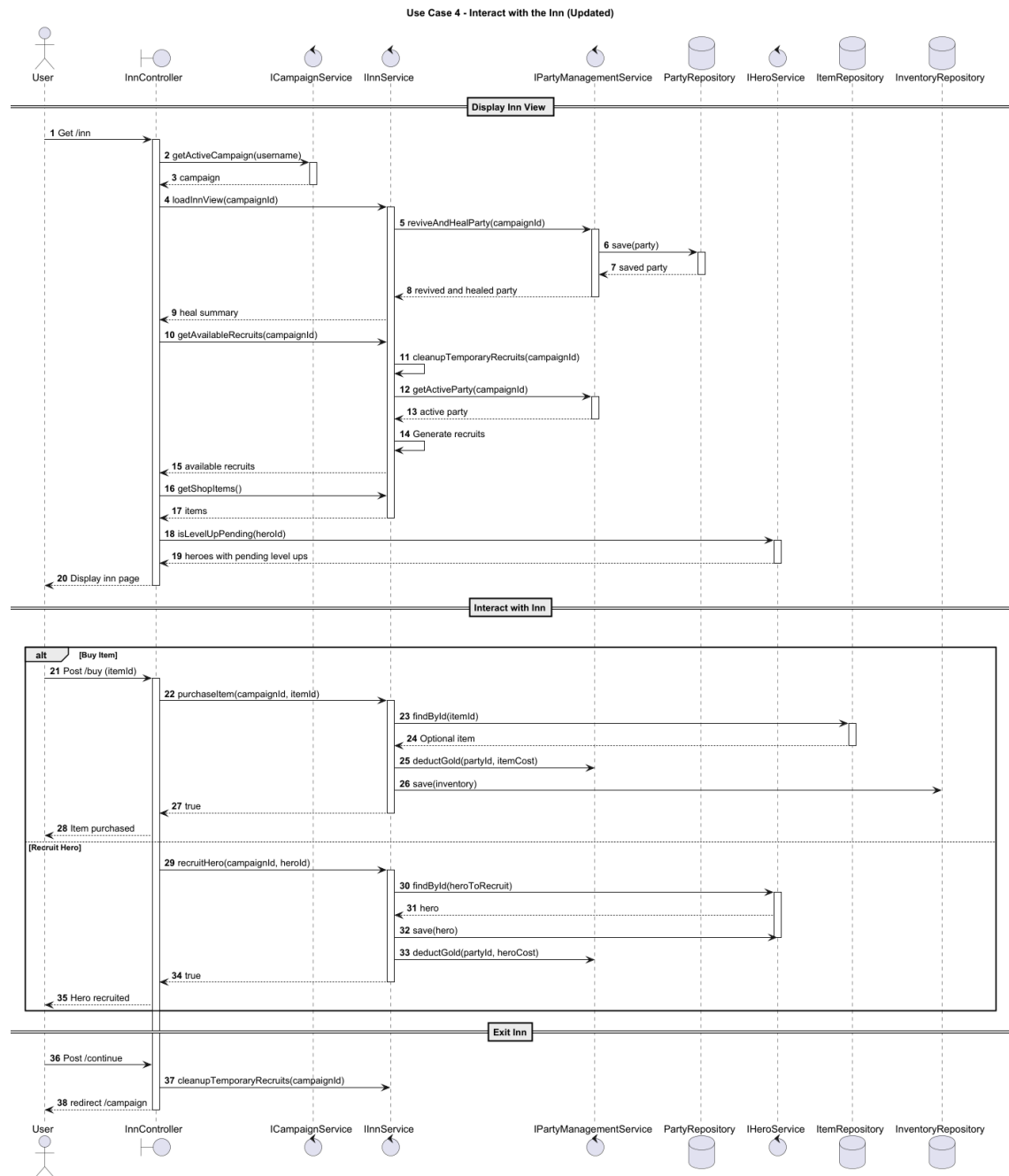
● Use Case 2: Start PvE Campaign



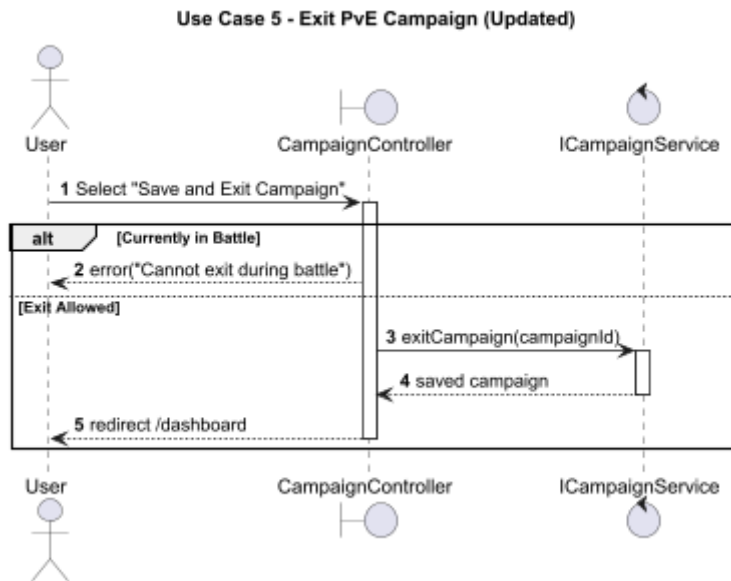
● Use Case 3: Single Battle



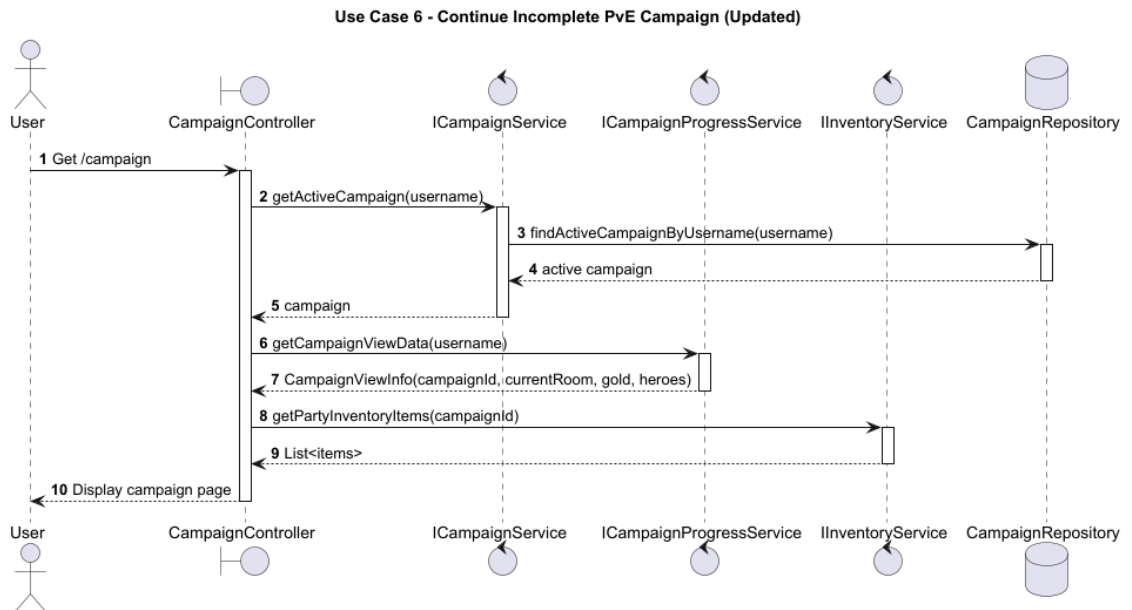
● Use Case 4: Interact with the Inn



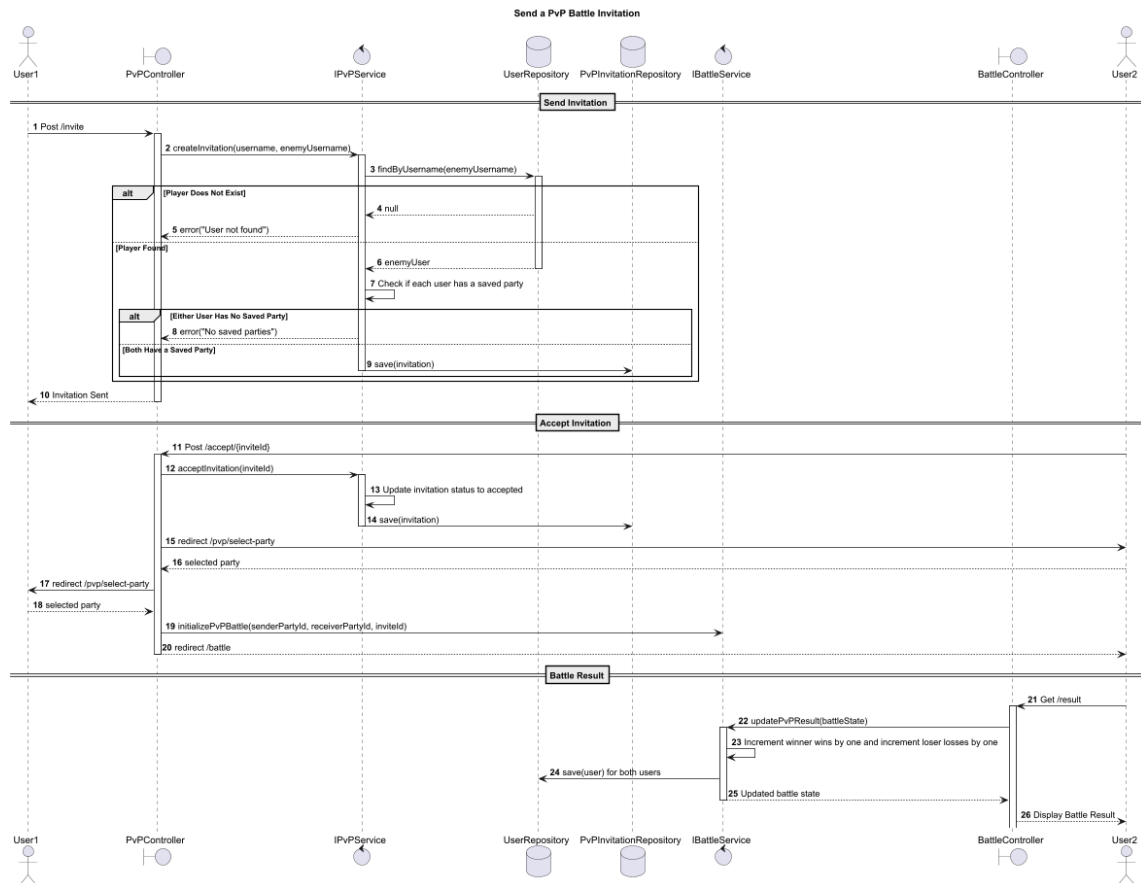
- Use Case 5: Exit PvE Campaign



- Use Case 6: Resume Campaign



- Use Case 7: Invite to PvP Battle



3. Major Design Decisions

The design decisions made for the Legends of Sword and Wand system were based on modularity, maintainability, extensivity and a clear separation of concerns.

The system follows an architecture that separates presentation, business logic and persistence concerns, enabling the development and testing of each, individual layer and component.

3.1. Layered Architecture

The system uses a layered architecture consisting of:

- Presentation Layer (Controllers)
- Application Layer (Services)
- Persistence Layer (Repositories)
- Domain Layer (Entities)

This design was selected to ensure high cohesion and low coupling.

Controllers handle user interaction only.

Services encapsulate business logic.

Repositories abstract database access.

Entities represent domain state.

This separation of concerns ensures that modifications to one layer do not affect the rest of the system.

3.2. Domain-Driven Modularization

The system is decomposed into modules aligned with core game domains:

- User Module
- Hero Module
- Party Module
- Battle Module
- Campaign Module
- PvP Module
- Persistence Module

Each module encapsulates a single responsibility. For example:

- The Battle Module handles only combat logic.
- The Campaign Module controls PvE progression but does not implement Battle mechanics.

This reduces cross-module dependencies and enforces modular growth.

3.3. Use of Interfaces

All services are defined through interfaces (e.g., IBattleService, ICampaignService).

Concrete implementations depend on abstractions, not on other concrete classes.

This achieves:

- Loose coupling
- Testability
- Easier future refactoring
- Clear dependency injection using Spring Boot

3.4. Strategy-Based Battle Execution

Battle actions are implemented using the Strategy pattern.

This decision prevents conditional explosion inside a single Battle class.

Each action is encapsulated in its own class implementing ActionStrategy.

This improves:

- Extensibility
- Readability
- Adherence to Open-Closed Principle

3.5. Database Abstraction with Spring Data JPA

The system uses Spring Data JPA repositories for persistence.

Repositories:

- Extend JpaRepository
- Contain no business logic
- Are injected into services

This design ensures:

- Database independence
- Simplified CRUD operations

- Proper separation of concerns

3.6. Prevention of God Classes

Special attention was paid to prevent large monolithic classes.

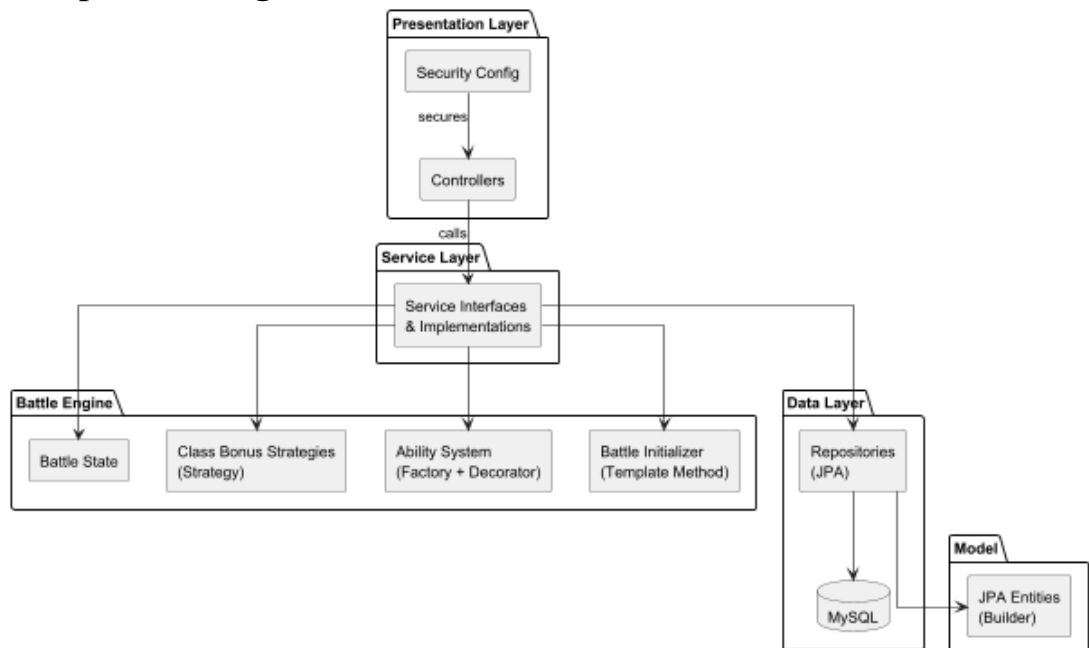
For example:

- Battle orchestration is separate from action execution.
- Campaign flow is separate from Battle resolution.
- Hero leveling is isolated from persistence logic.

This improves maintainability and readability.

4. Architecture

4.1. Component Diagram



4.2.

Component Layer	Description
Presentation Layer	Handles User registration, authentication and profile persistence.
	Provides user profile data, Hall of Fame rankings and PvP standings.
Service Layer	Manages Hero creation, leveling, class bonuses and hybrid class resolution.

	Manages Party composition, Hero membership and gold tracking.
	Controls PvP flow, including invitation creation, acceptance and Battle interactions.
	Controls PvE flow, room progression, score calculation and Campaign persistence.
	Manages Inn interactions, including Hero regeneration, Item purchases and Hero recruitment.
	Manages the party Item inventory, Item usage and inventory persistence.
Battle Engine	Implements the Battle engine, turn logic, action execution and reward distribution.
Data Layer	Uses JPA repositories to perform CRUD operations and manage entity lifecycle.
	Provides data access to Service Layer components such as PvP, PvE, Party, and Inventory services.
	Handles persistence and database interaction for the system.
Module	Implements the Builder pattern for flexible and controlled object construction.
	Defines relationships between entities (e.g., Hero, Party, Items, Battles).
	Serves as the shared data structure used across Service and Data layers.

4.3. Interface Specification

<i>Interface Name</i>	<i>Signature of Operations Exposed in Interface</i>	<i>Description of the Operation</i>
IAuthService	User register(RegisterRequest request)	Creates a new User profile, encodes their password and persists it to the database.
IUserService	DashboardInfo getDashboardInfo(String username)	Returns dashboard data including active Campaign and Party summaries for the User.
	Long getUserIdByUsername(String username)	Looks up and returns the database ID for the given username.
IProfileService	ProfileInfo getProfile(String username)	Retrieves full profile data including Hero history and Campaign results.
	List<HallOfFameEntry> getHallOfFame()	Returns the top Campaign scores across all Users.
	List<PvPStandingEntry> getPvPStandings()	Returns the PvP win/loss leaderboard for all players.
IHeroService	void createBaseHeroForParty(Long partyId, String heroName, HeroClass heroClass)	Instantiates a Hero with base stats, assigns the selected class and adds it to the Party.
	Hero levelUp(Long heroId, HeroClass chosenClass)	Applies level growth and class bonuses to the Hero using the chosen class.

	Hero addExperience(Long heroId, int amount)	Adds experience points to the Hero and flags a level-up if the threshold is met.
	boolean isLevelUpPending(Long heroId)	Returns true if the Hero has accumulated enough XP to level up but has not yet done so.
	Optional<Hero> findById(Long heroId)	Retrieves a Hero entity by its database ID.
	Hero save(Hero hero)	Persists the Hero entity to the database.
	void delete(Long heroId)	Removes the Hero record from the database.
IPartyService	Party createPartyForUser(Long userId)	Creates and persists a new empty Party linked to the specified User.
	void deleteParty(Long partyId)	Removes the Party and its associated Hero memberships from the database.
IPartyManagementService	Party getActiveParty(Long campaignId)	Retrieves the Party currently associated with the active Campaign.
	Party reviveAndHealParty(Long campaignId)	Restores all Heroes in the Campaign Party to full health and mana.
	List<String> reviveAndHealPartyWithSummary(Long campaignId)	Revives and heals the Party and returns a list of log messages describing the changes.

	void deductGold(Long partyId, int amount)	Subtracts the specified amount of gold from the Party's balance.
	void addGold(Long partyId, int amount)	Adds the specified amount of gold to the Party's balance.
	void addHeroToParty(Long partyId, Long heroId)	Associates an existing Hero with the specified Party.
IBattleService	BattleState initializePvEBattle(Long campaignId, int playerCumulativeLevel)	Creates a PvE BattleState with generated enemies scaled to the party's cumulative level.
	BattleState initializePvPBattle(Long senderPartyId, Long receiverPartyId, Long invitationId)	Creates a PvP BattleState between two player Parties.
	BattleState executePlayerAction(BattleState state, ActionType actionType, Long targetBattleId, Integer abilityIndex)	Executes the player's chosen action (attack, ability, or item use) against the target.
	BattleState executeEnemyTurn(BattleState state)	Selects and executes the enemy's action for their turn.
	BattleStatus checkBattleStatus(BattleState state)	Evaluates and returns the current win/loss/ongoing status of the Battle.
	Map<String, Object> awardBattleRewards(BattleState state)	Distributes gold and XP to the winning Party's Heroes after a Battle victory.

	void applyBattleLoss(BattleState state)	Applies defeat consequences, such as health reduction, to the losing Party.
	void updatePvPResult(BattleState state)	Persists the PvP win/loss outcome for both participating players.
IInnService	List<String> loadInnView(Long campaignId)	Returns a summary of the Campaign Party's current status for display at the Inn.
	List<Item> getShopItems()	Retrieves all Items available for purchase in the Inn shop.
	List<Hero> getAvailableRecruits(Long campaignId)	Returns recruitable Heroes generated for the current Campaign's Inn visit.
	boolean purchaseItem(Long campaignId, Long itemId)	Deducts gold and adds the Item to the Campaign Party's inventory if the Party can afford it.
	boolean recruitHero(Long campaignId, Long heroId)	Adds a recruit to the Campaign Party if sufficient gold is available.
	void cleanupTemporaryRecruits(Long campaignId)	Removes any un-recruited temporary Heroes generated for the Inn visit.
IInventoryService	Inventory saveInventory(Inventory inventory)	Persists an Inventory record to the database.

	boolean useItem(Long campaignId, Long heroId, Long itemId)	Applies the Item's effect to the target Hero and removes it from the Party's inventory.
	List<Item> getPartyInventoryItems(Long campaignId)	Returns all Items currently held in the Campaign Party's inventory.
IPvPService	void createInvitation(String senderUsername, String receiverUsername)	Creates and persists a PvP invitation from the sender to the receiver.
	void acceptInvitation(Long inviteId)	Marks the specified PvP invitation as accepted, enabling the Battle to be initiated.
ICampaignService	Campaign startNewCampaign(String username, String heroName, HeroClass heroClass)	Initializes a new PvE Campaign with a starting Hero and persists it to the database.
	Campaign getActiveCampaign(String username)	Retrieves the User's currently active Campaign.
	boolean hasActiveCampaign(Long userId)	Returns true if the User currently has an active Campaign in progress.
	Campaign exitCampaign(String username)	Pauses the active Campaign and persists its state so it can be resumed later.
	Campaign completeCampaign(String username)	Marks the Campaign as completed and finalizes its state.

	void abandonCampaign(String username)	Permanently discards the active Campaign without saving progress.
	void savePartyFromCampaign(Long campaignId, Long userId)	Persists the Campaign Party as a new saved Party for the User.
	void replacePartyFromCampaign(Long campaignId, Long userId, Long partyIdToReplace)	Replaces an existing saved Party with the Campaign Party.
	int getPartyCumulativeLevel(String username)	Returns the sum of all Hero levels in the active Campaign Party.
ICampaignProgress Service	RoomType enterNextRoom(String username)	Advances the Campaign to the next room and returns its type (Battle or Inn).
	boolean isCampaignComplete(String username)	Returns true if all Campaign rooms have been completed.
	int calculateAndPersistScore(Campaign campaign)	Computes the final Campaign score based on performance and saves it to the database.
	CampaignViewInfo getCampaignViewData(String username)	Returns current Campaign state data for rendering the Campaign UI.
	CompleteCampaignInfo getCompletionData(String username)	Returns summary data to be displayed on the Campaign completion screen.

	void clearRoomPending(String username)	Resets the pending room flag after the current room has been fully resolved.
--	--	--

4.4. Architectural Patterns Used

The system follows a layered architectural pattern consisting of Presentation, Application and Persistence layers. This pattern was selected to ensure clear separation of concerns, maintainability and scalability. Controllers handle incoming requests, service interfaces define the application's business logic boundaries, service implementations coordinate domain operations and repositories manage database persistence using Spring Data JPA with MySQL. This structure promotes modularity and ensures that changes in one layer do not directly affect others.

Additionally, the system adopts a modular service-oriented structure where each functional area (User, Hero, etc.) is encapsulated within its own module and exposes a single service interface. This approach supports loose coupling between modules and enables reuse of the Battle module by both Campaign and PvP components. By enforcing dependency on interfaces rather than concrete implementations, the design adheres to the Dependency Inversion Principle, improving testability and architectural clarity.

4.5. Database Configuration

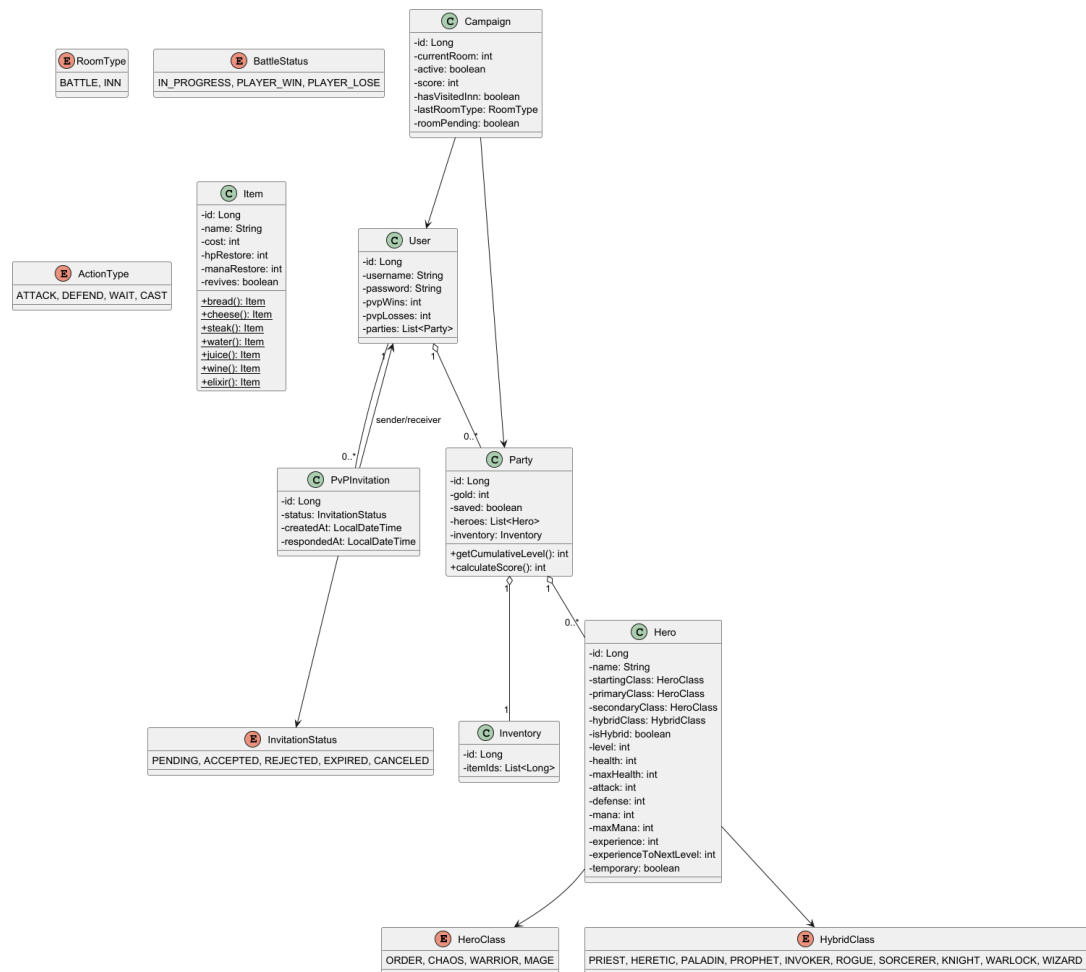
The application supports two runtime profiles. The dev profile connects to a MySQL database configured via environment variables (DB_URL, DB_USER, DB_PASS). The demo profile uses an in-memory H2 database pre-seeded with two user accounts (player1 and player2) and their saved parties, allowing the application to be run without any external database setup. The active profile is controlled by the spring.profiles.active property in application.properties.

This profile-based approach satisfies the project requirement that the application should work with any database backend — the service and repository layers are database-agnostic, with only the Spring profile configuration changing between environments.

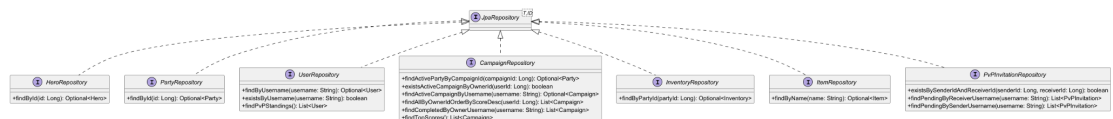
5. Detailed Class Diagrams

5.1. UML Class Diagrams

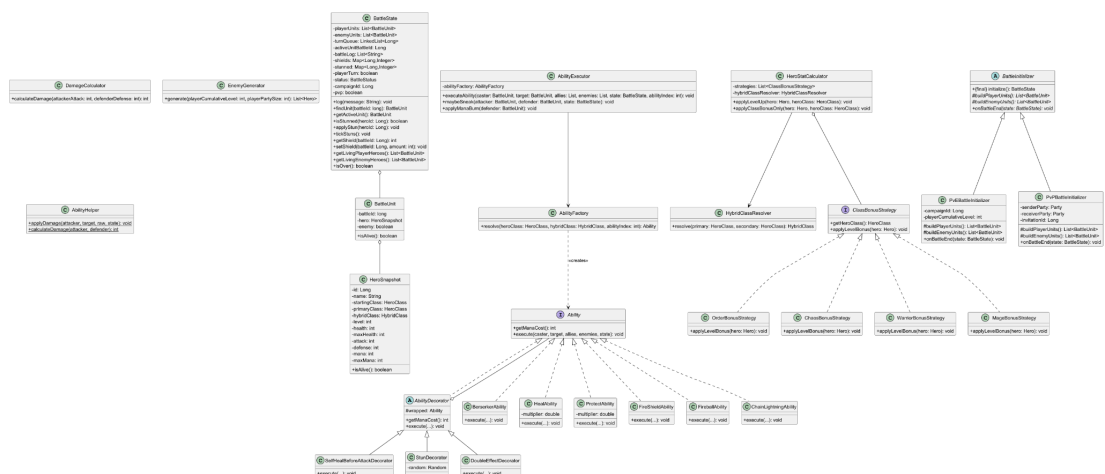
Model Component



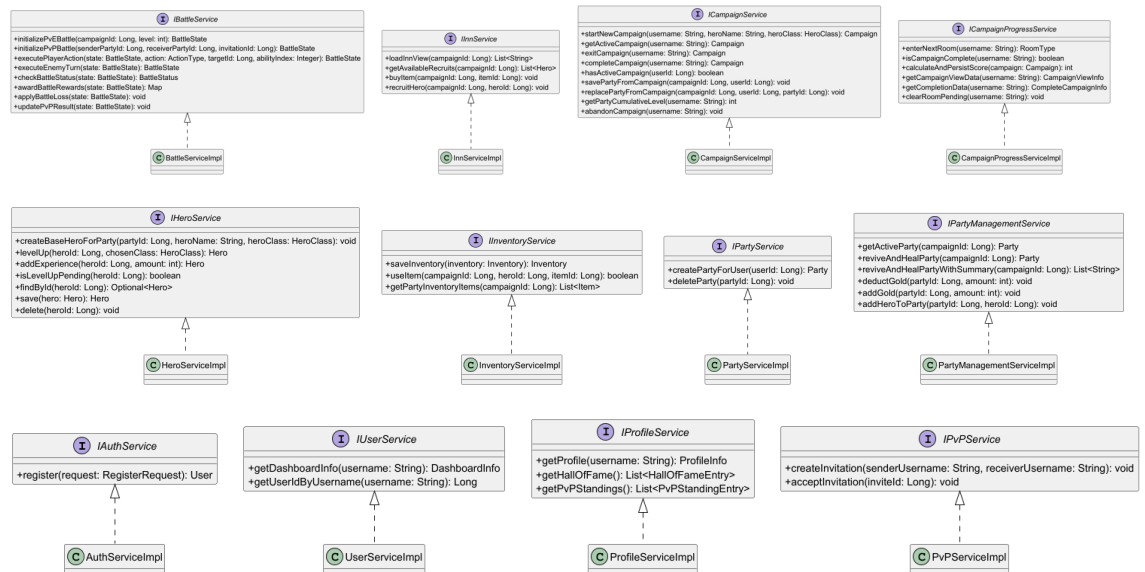
Repository Component



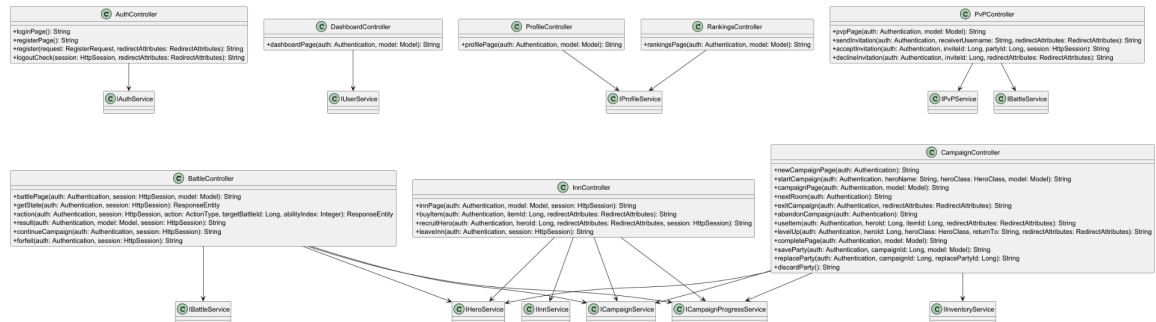
Battle Engine



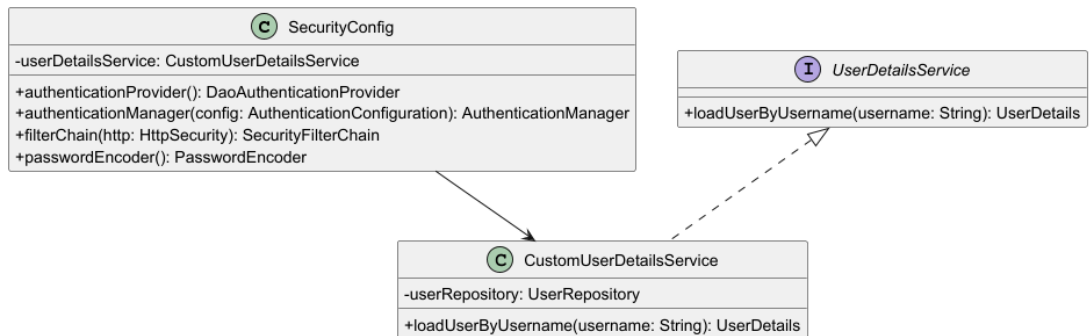
Service Layer Component



Controller Component



Config Component



6. Use of Design Patterns

The following six design patterns are implemented in the system. Each pattern is documented with its location in the codebase, the classes involved, its purpose, and why it was chosen.

6.1. Strategy Pattern

Location: battle/strategy/

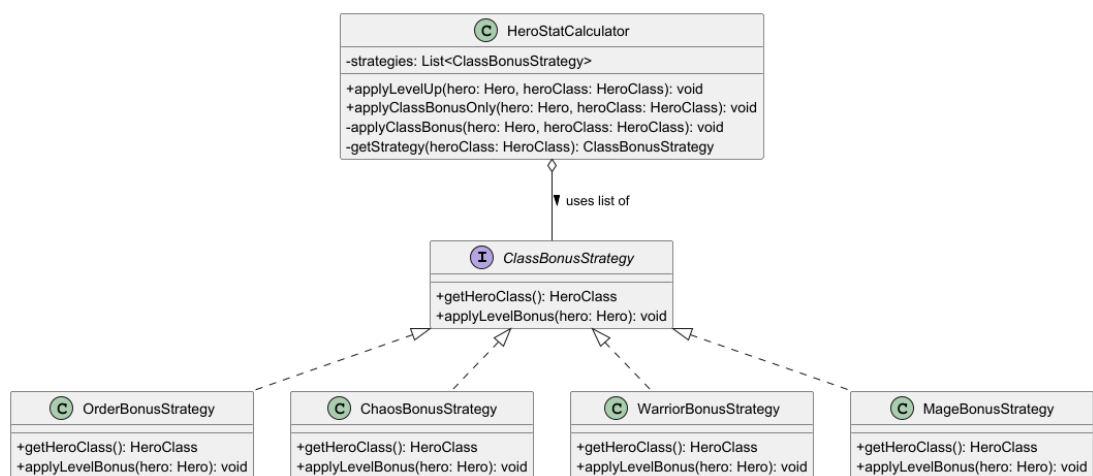
Classes:

- ClassBonustrategy (interface)

- OrderBonusStrategy, ChaosBonusStrategy, WarriorBonusStrategy, MageBonusStrategy (concrete implementations)
- HeroStatCalculator (context)

Description: Each hero class has a distinct set of stat bonuses applied on level-up. Rather than encoding all four sets of bonuses in a single conditional block inside HeroStatCalculator, each class's bonus logic is encapsulated in its own ClassBonusStrategy implementation. HeroStatCalculator receives the full list of strategies via Spring dependency injection and selects the correct one at runtime by matching on HeroClass.

Why Chosen: Eliminates a large if-else or switch block in HeroStatCalculator. Adding a new hero class requires only a new strategy class with no modification to existing code, satisfying the Open/Closed Principle. Each strategy is also independently testable.



6.2. Factory Pattern

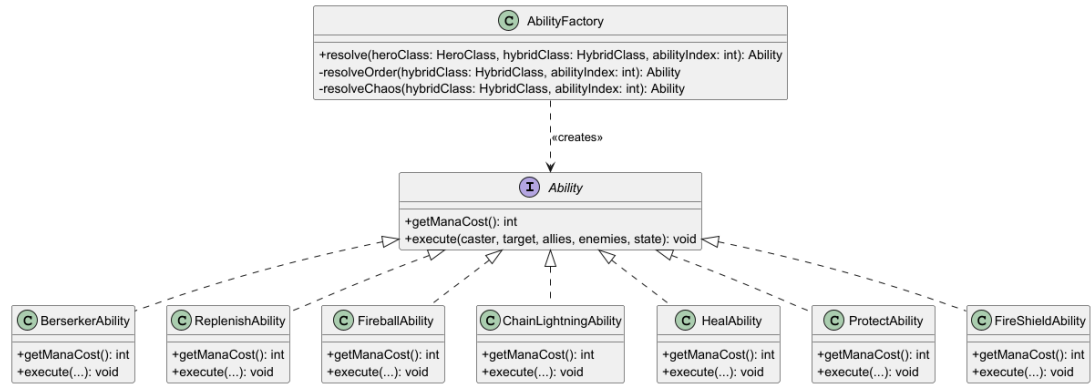
Location: battle/ability/AbilityFactory.java

Classes:

- AbilityFactory (factory)
- Ability (product interface)
- ProtectAbility, HealAbility, FireShieldAbility, FireballAbility, ChainLightningAbility, BerserkerAbility, ReplenishAbility (concrete products)

Purpose: AbilityFactory.resolve() is the single point of construction for all Ability instances. Given a hero's HeroClass, HybridClass and ability slot index, it determines and returns the correct concrete ability – including stacking any Decorator wrappers for hybrid class effects (see Section 7.4). Callers such as AbilityExecutor never instantiate abilities directly.

Why Chosen: Centralises all ability construction and hybrid resolution logic in one place. Adding a new ability of hybrid modifier requires changes only to AbilityFactory, with no impact on service or controller code. Registered as a Spring @Component for dependency injection.



6.3. Template Method Pattern

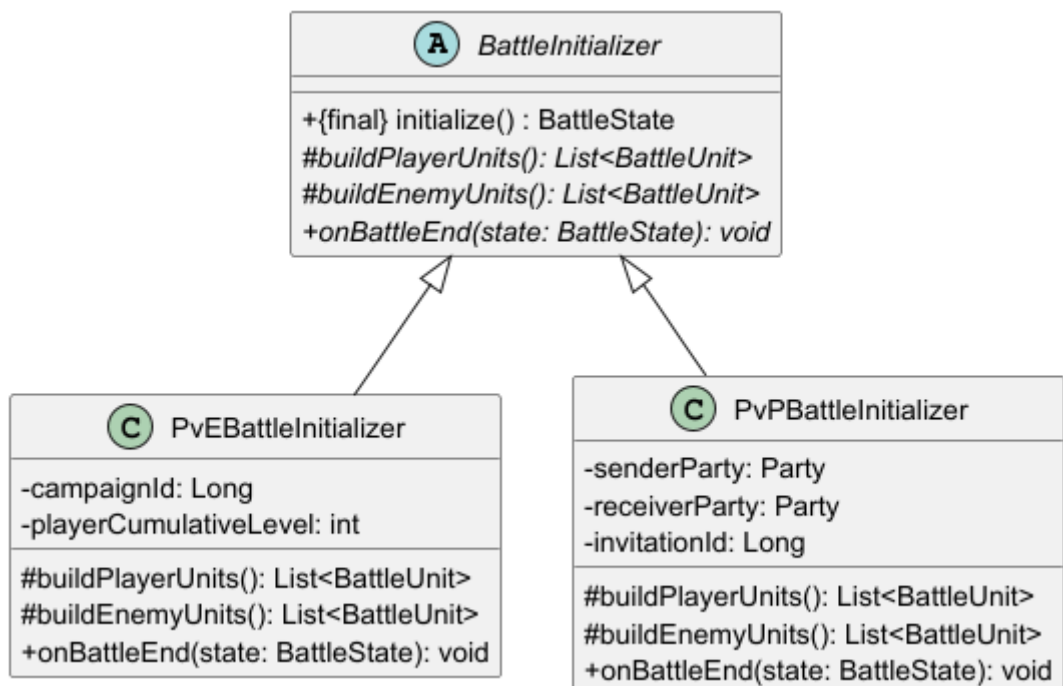
Location: battle/initializer

Classes:

- BattleInitializer (abstract class defining the template)
- PvEBattleInitializer, PvPBattleInitializer (concrete subclasses)

Description: BattleInitializer defines a final method initialize() that specifies the fixed skeleton for setting up any battle: build player units → build enemy units → sort all units by level and attack → populate the turn queue → set the first active unit. The abstract hook methods buildPlayerUnits(), buildEnemyUnits() and onBattleEnd() are left for subclasses to implement. PvEBattleInitializer sources player units from the active campaign party and generates scaled enemies via EnemyGenerator. PvPBattleInitializer sources both sides from saved parties and handles win/loss record updates on battle end.

Why Chosen: Ensures every battle type follows an identical, predictable initialisation sequence while allowing PvE and PvP to differ in how they source units and handle post-battle logic. The final keyword on initialize() prevents subclasses from accidentally breaking the sequence.



6.4. Decorator Pattern

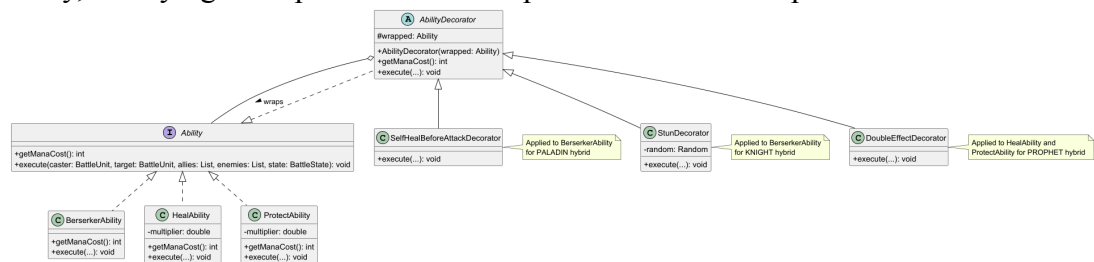
Location: battle/ability/decorator

Classes:

- AbilityDecorator (abstract decorator)
- SelfHealBeforeAttackDecorator, StunDecorator, DoubleEffectDecorator (concrete decorators)
- Ability (component interface)

Description: Hybrid class effects modify the behaviour of base abilities without altering the base ability classes themselves. AbilityDecorator implements Ability and wraps another Ability, delegating all calls to it by default. Concrete decorators override execute() to add behaviour before or after the wrapped ability fires. SelfHealBeforeAttackDecorator heals the caster for 10% max HP before the attack (Paladin hybrid). StunDecorator gives a 50% chance to stun the primary target after the attack (Knight hybrid). DoubleEffectDecorator doubles the effect magnitude of HealAbility and ProtectAbility (Prophet hybrid). Decorators are stacked by AbilityFactory at construction time and are fully interchangeable with base abilities since they share the Ability interface.

Why Chosen: Hybrid class effects are open-ended combinations of base abilities and modifiers. Subclassing each combination would result in a combinatorial explosion of classes (e.g. StunningBerserkerAbility, HealingBerserkerAbility). Decorator allows any modifier to be applied to any base ability independently and composed freely, satisfying the Open/Closed Principle without subclass proliferation.



6.5. Facade Pattern

Location: service/battle/IBattleService.java and service/battle/impl/BattleServiceImpl.java

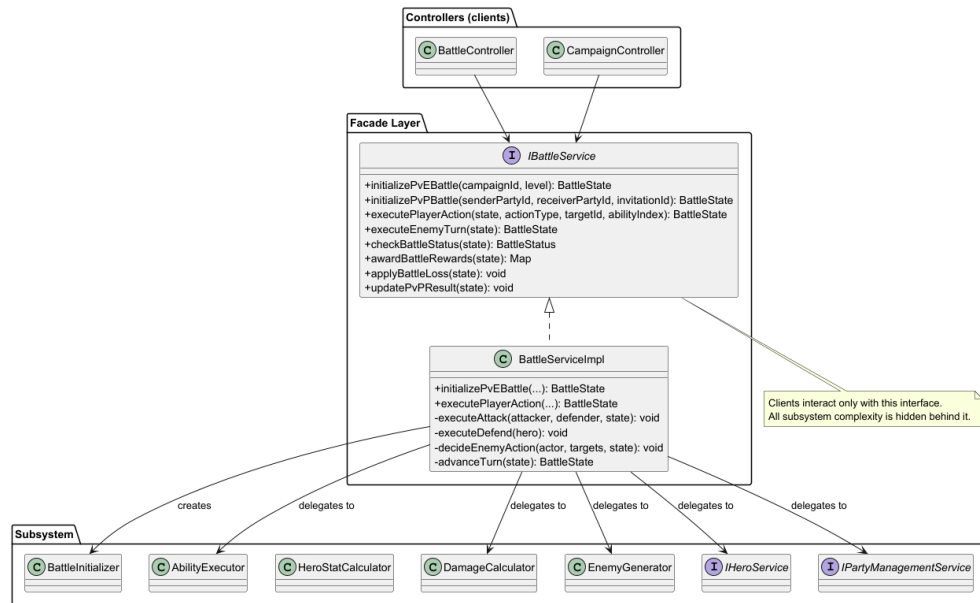
Classes:

- IBattleService (facade interface)
- BattleServiceImpl (facade implementation)
- BattleController, CampaignController (clients)

Description: The battle subsystem involves numerous collaborating components: BattleInitializer, AbilityExecutor, DamageCalculator, EnemyGenerator, HeroStatCalculator, IHeroService, and IPartyManagementService. IBattleService exposes a clean, cohesive 8-method interface covering the full battle lifecycle — initialisation, turn execution, enemy AI, status checking, reward distribution, and loss penalties. Controllers interact only with this interface and have no knowledge of the subsystem components behind it.

Why Chosen: Shields controllers from the complexity of battle orchestration. Any change to internal battle logic — such as modifying enemy AI or reward calculation — requires no changes to controllers. Also supports the GRASP principle of

Information Expert by keeping battle knowledge inside the battle service rather than leaking it into controllers.



6.6. Builder Pattern

Location: model/Item.java

Classes:

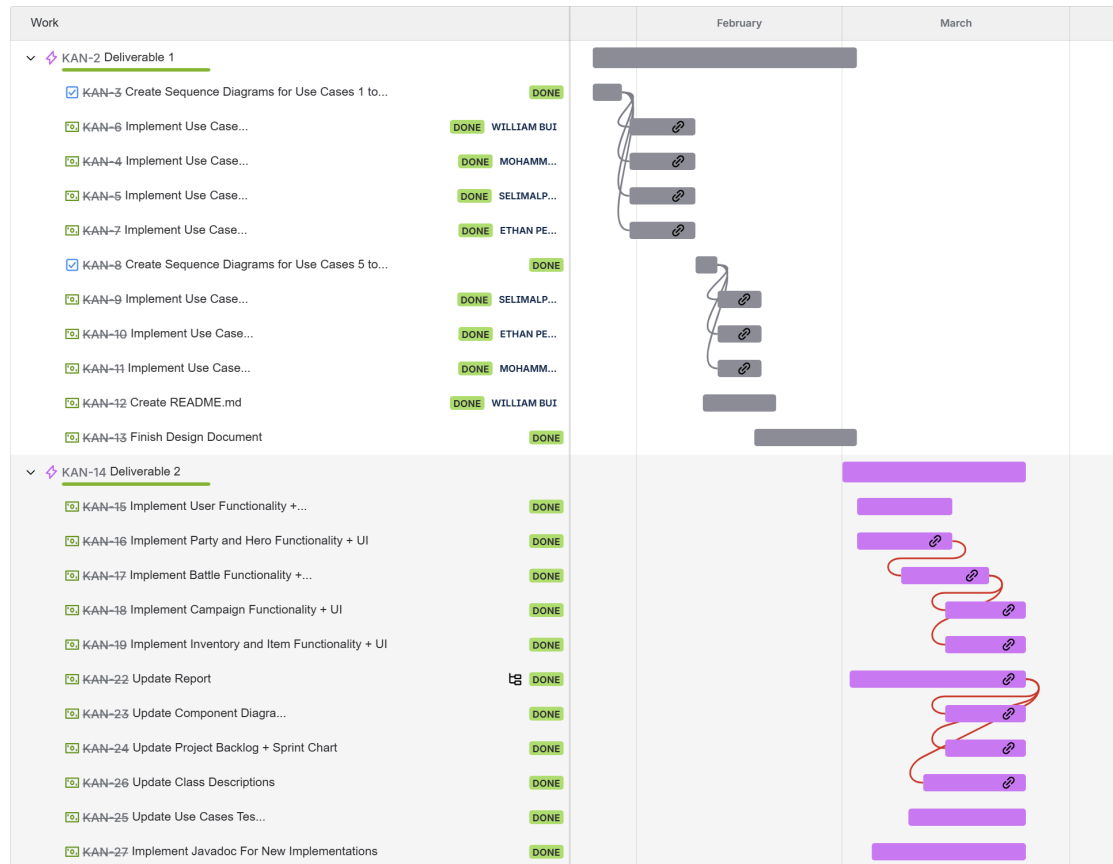
- Item (product)
- Item.ItemBuilder (builder, generated by Lombok @Builder)

Description: Item uses Lombok's @Builder annotation to generate a fluent builder for constructing item instances. Rather than exposing the builder directly to all callers, Item provides a set of static factory methods — bread(), cheese(), steak(), water(), juice(), wine(), and elixir() — each of which uses the builder internally to produce a fully configured Item with the correct name, cost, HP restore, mana restore, and revival flag as defined in the game specification. Callers receive a ready-made item without needing to know which fields are relevant for each item type.

Why Chosen: Item construction involves multiple optional fields (hpRestore, manaRestore, revives) that vary per item type. The Builder pattern avoids telescoping constructors and makes it impossible to accidentally construct a malformed item. The static factory methods further simplify the API for callers such as InnServiceImpl, which needs to offer items to the player without re-specifying their properties each time.

7. Activities Plan

7.1. Gantt Chart



7.2. Project Backlog and Sprint Backlog

Project BacklogSprint Backlog (Jan 29, 2026 - Feb 20, 2026)

ID	Task Description	Deliverable	Due Date	Status	Assigned To
KAN-3	Create Sequence Diagrams for Use Cases 1 to 4	Deliverable 1	Jan 29, 2026	Done	Team
KAN-4	Implement Use Case 2	Deliverable 1	Feb 8, 2026	Done	Mohammed
KAN-5	Implement Use Case 3	Deliverable 1	Feb 8, 2026	Done	Selim
KAN-6	Implement Use Case 1	Deliverable 1	Feb 8, 2026	Done	Will
KAN-7	Implement Use Case 4	Deliverable 1	Feb 8, 2026	Done	Ethan
KAN-8	Create Sequence Diagrams for Use Cases 5-7	Deliverable 1	Feb 11, 2026	Done	Team
KAN-9	Implement Use Case 5	Deliverable 1	Feb 17, 2026	Done	Selim
KAN-10	Implement Use Case 6	Deliverable 1	Feb 17, 2026	Done	Ethan
KAN-11	Implement Use Case 7	Deliverable 1	Feb 17, 2026	Done	Mohammed
KAN-12	Create README.md	Deliverable 1	Feb 19, 2026	Done	Will
KAN-13	Finish Design Document	Deliverable 1	Mar 2, 2026	Done	Team
KAN-15	Implement User Functionality + UI	Deliverable 2	Mar 15, 2026	To Do	Unassigned
KAN-16	Implement Party and	Deliverable 2	Mar 15, 2026	To Do	Unassigned

ID	Task Description	Deliverable	Due Date	Status	Assigned To
	Hero Functionality + UI				
KAN-17	Implement Battle Functionality + UI	Deliverable 2	Mar 20, 2026	To Do	Unassigned
KAN-18	Implement Campaign Functionality + UI	Deliverable 2	Mar 29, 2026	To Do	Unassigned
KAN-19	Implement Inventory and Item Functionality + UI	Deliverable 2	Mar 29, 2026	To Do	Unassigned

Task ID	Task Description	Due Date	Status	Assigned To
KAN-3	Create Sequence Diagrams for Use Cases 1–4	Jan 29, 2026	Done	Team
KAN-4	Implement Use Case 2	Feb 8, 2026	Done	Mohammed
KAN-5	Implement Use Case 3	Feb 8, 2026	Done	Selim
KAN-6	Implement Use Case 1	Feb 8, 2026	Done	Will
KAN-7	Implement Use Case 4	Feb 8, 2026	Done	Ethan
KAN-8	Create Sequence Diagrams for Use Cases 5–7	Feb 11, 2026	Done	Team
KAN-9	Implement Use Case 5	Feb 17, 2026	Done	Selim
KAN-10	Implement Use Case 6	Feb 17, 2026	Done	Ethan
KAN-11	Implement Use Case 7	Feb 17, 2026	Done	Mohammed
KAN-12	Create README.md	Feb 19, 2026	Done	Will
KAN-13	Finish Design Document	Mar 2, 2026	In Progress	Team

Project BacklogSprint Backlog (Feb 20, 2026 - March 25, 2026)

ID	Task Description	Due Date	Status	Assigned To
KAN-13	Finish Design Document	Mar 2, 2026	Done	Team
KAN-15	Implement User Functionality + UI	Mar 15, 2026	Done	Unassigned
KAN-16	Implement Party and Hero Functionality + UI	Mar 15, 2026	Done	Unassigned
KAN-17	Implement Battle Functionality + UI	Mar 20, 2026	Done	Unassigned
KAN-18	Implement Campaign Functionality + UI	Mar 29, 2026	Done	Unassigned
KAN-19	Implement Inventory and Item Functionality + UI	Mar 29, 2026	Done	Unassigned
KAN-22	Update Report	March 25, 2026	Done	Team
KAN-23	Update Component Diagram	March 25, 2026	Done	Mohammed & Ethan

ID	Task Description	Due Date	Status	Assigned To
KAN-24	Update Project Backlog + Sprint Chart	March 25, 2026	Done	Ethan
KAN-26	Update Class Descriptions	March 25, 2026	Done	Team
KAN-25	Update Use Case Tests	March 25, 2026	Done	Will
KAN-27	Implement Javadoc For New Implementations	March 25, 2026	Done	Ethan

7.3. Group Meeting Logs

Present Group Members	Meeting Date	Issues Discussed / Resolved
Ethan, Mohammed, Will, Selim	January 27, 2026	- Discuss resources needed for project; what to install, setup - Discuss project deliverable 1 requirements - Began configuring mySQL - Set up next meeting
Ethan, Mohammed, Will, Selim	January 28, 2026	- Set up mySQL database connection to intelliJ
Ethan, Mohammed, Will, Selim	February 2, 2026	- Discussed work delegation - Discussed including Javadoc
Ethan, Mohammed, Will, Selim	February 10, 2026	- Discuss where everyone is at in their implementation - Assigned more work to people (more use cases)
Ethan, Mohammed, Will, Selim	February 11, 2026	- Discussed how we will make our component diagram - We each work on what we have implemented then put it all together
Ethan, Mohammed, Will	February 18, 2026	- Discussed our priorities for deliverable 1 - Decided to focus more on having a polished report more importantly than having actual code implementation - Aim to change our sequence diagrams to more accurately represent our design - Ensure when implementing to the main classes of each use case,

		implement detailed exception handling reports - Discuss how we are going to make the component diagram - We will probably all get together and work on it together - Discussed design patterns used to add to our report - Created a rough component diagram to give a better idea of what the fleshed out version would look like - Worked on adding the design patterns we used in our code, to the report
Ethan, Mohammed, Will, Selim	February 26, 2026	- Discussed completed component diagram + class diagram - Discussed report progress - Discussed work delegation for more detailed code implementation - Discussed adding runnable tests to the use cases for deliverable 1 - Discussed how we are going to implement our Gantt chart and project/sprint backlogs
Ethan, Mohammed, Will, Selim	March 10, 2026	- Tested new implementation for running the application - Review frontend application for project & how backend connects to it using thymeleaf - Discussed work delegation for deliverable 2
Ethan, Mohammed, Will	March 11, 2026	- Discussed what needs to be added on the backend - Hero stat calculator - Which design principals we're going to implement - How we're going to implement different classes + multiclassing/hybrid class - Worked on hero stat calculator together

8. Test Driven Development

Access URL: localhost:8080/login

USE CASE 1

Test ID	TC-01
----------------	-------

Category	Authentication – Profile Creation
Requirements Coverage	UC1 – Create a profile and log in
Initial Condition	The application is running. No account with username “testuser” exists in database.
Procedure	1. Navigate to /register. 2. Enter username testuser and a valid password. 3. Click “BEGIN YOUR LEGEND”. 4. Navigate to /login. 5. Enter the same credentials. 6. Click “ENTER THE REALM”.
Expected Outcome	Registration succeeds and redirects to /login. Login succeeds and redirects to /dashboard. The dashboard displays the username testuser in the top left corner.
Notes	Username must be unique. Attempting to register with an already existing username should display an error and not create a duplicate account.

Test ID	TC-02
Category	Campaign – PvE Mode
Requirements Coverage	UC2 – Start a new PvE campaign
Initial Condition	User is logged in. User has no active campaign.
Procedure	1. Navigate to /campaign/new by pressing the “NEW CAMPAIGN” button. 2. Enter hero name Odin. 3. Select class CHAOS. 4. Click “BEGIN THE CAMPAIGN”.
Expected Outcome	the system detects duplicate username registration is rejected an error message is displayed no new user is created
Notes	Hero name must not be blank. Starting stats must match the specification: 5 attack, 5 defense, 100 HP, 50 mana at level 1, plus 3 attack and 5 HP per Chaos level -> Total: 8 attack, 5 defense, 105 HP, 50 mana.

Test ID	TC-03
Category	Battle System – Basic Attack

Requirements Coverage	UC3 – Go through a single battle
Initial Condition	User is logged in with an active campaign. User is on the campaign page.
Procedure	1. Click “ENTER NEXT ROOM” until a battle room is entered. 2. On the battle page, note the enemy’s current HP, attack and defense values. 3. Select ATTACK action on any enemy target. 4. Confirm the action.
Expected Outcome	The enemy’s HP decreases by $\max(1, \text{playerAttack} - \text{enemyDefense})$. The battle log shows the attack with correct damage value. Turn advances to the next unit.
Notes	Damage formula is $\text{attacker.attack} - \text{defender.defense}$ with a minimum of 1. Shield absorption should reduce damage if a shield is active.

Test ID	TC-04
Category	Battle System – Ability Cast
Requirements Coverage	UC3 – Go through a single battle
Initial Condition	User is logged in. User starts a new campaign with a Chaos class hero. User has entered a battle room. The Chaos hero is the active unit with 50 mana. At least one enemy is present.
Procedure	1. On the battle page, select Fireball action on any enemy target. 2. Confirm the action.
Expected Outcome	Primary target and up to two additional enemies each take calculated damage. Caster’s mana decreases by 30, leaving 20 mana remaining. All hits appear in the battle log. Turn advances to the next unit.
Notes	If fewer than three enemies are present, only the available enemies are hit. A Warrior hero cannot be used for this test case as Berserker Ability costs 60 mana which exceeds the level 1 maximum of 50.

USE CASE 2

Test ID	TC-05
Category	Battle System – Defend
Requirements Coverage	UC3 – Go through a single battle
Initial Condition	User is in an active battle. Active hero has less than maximum HP or mana.
Procedure	1. Note the active hero's current HP and mana. 2. Select DEFEND action. 3. Confirm the action.
Expected Outcome	Hero's HP increases by 10 (capped at max HP). Hero's mana increases by 5 (capped at max mana). Turn advances. Battle log shows the defend action.
Notes	If hero is already at max HP and max mana, both values remain unchanged at their maximums.

Test ID	TC-06
Category	Inn – Interaction
Requirements Coverage	UC4 – Interact with the inn
Initial Condition	User is logged in with an active campaign. User has entered an inn room. Party has at least one hero with less than full HP and/or mana. Party has at least 200 gold. Party has fewer than 5 heroes. Campaign is within first 10 rooms.
Procedure	1. Verify all heroes are shown as fully healed on the inn page. 2. Purchase one Water item (150g). 3. Verify gold decreased by 150 and Water appears in inventory. 4. If a recruit is available, click Recruit on a level 1 hero (free) or a higher level hero if user has sufficient gold. 5. Click "CONTINUE TO NEXT ROOM" to leave inn.
Expected Outcome	All heroes are at full HP and mana on arrival. Water is added to inventory. Gold decreases correctly. Recruited hero appears in the party. Clicking "CONTINUE TO NEXT ROOM" returns to the campaign page with room counter incremented.
Notes	Heroes are revived and fully healed automatically on inn entry. Recruiting costs 200g per level for heroes above level 1. Party cannot exceed 5 heroes.

Test ID	TC-07
Category	Campaign – Save and Resume
Requirements Coverage	UC5 – Exit campaign / UC6 – Continue incomplete PvE campaign
Initial Condition	User is logged in with an active campaign. User is on the campaign page (not in a battle).
Procedure	1. Note the current room number, hero stats and gold. 2. Click “SAVE AND EXIT CAMPAIGN”. 3. Verify redirect to dashboard. 4. Click “CONTINUE” from the dashboard. 5. Navigate to /campaign.
Expected Outcome	Campaign page reloads with the same room number, hero stats and gold as before exit. Database retains all campaign state.
Notes	Exit must be blocked if a battle room is currently pending. Attempting to exit during a battle should redirect to the current battle page.

Test ID	TC-08
Category	Hero Management – Level Up
Requirements Coverage	UC2 – PvE campaign (party management)
Initial Condition	User has won a battle. At least one surviving hero has enough experience to level up (experience >= experienceToNextLevel).
Procedure	1. On the battle result page, locate the level-up panel for the eligible hero. 2. Select class CHAOS for the level-up. 3. Click Level Up.
Expected Outcome	Hero's level increases by 1. Stats increase by base gains (+1 attack, +1 defense, +5 HP, +2 mana) plus Chaos bonus (+3 attack, +5 HP). New experienceToNextLevel is calculated correctly. Level-up panel disappears after confirmation.
Notes	If no hero has enough experience, the level-up panel should not appear. Hybrid class bonuses apply if the hero has reached level 5 in two classes.

USE CASE 3

Test ID	TC-09
Category	Campaign – Completion
Requirements Coverage	UC2 – Start a new PvE campaign (completion flow)
Initial Condition	User is logged in with an active campaign at room 29/30. Last room has been completed (no pending room).
Procedure	1. On the campaign page showing room 29/30, click “ENTER NEXT ROOM”. 2. Complete the resulting battle or inn room. 3. On the battle result or inn page, click Continue.
Expected Outcome	Redirects to /campaign/complete. Completion page shows the calculated score (hero levels $\times$ 100 + gold $\times$ 10), final gold, and hero list. Options to save party, replace a saved party, or discard are shown.
Notes	Score calculation must match the spec. If the player already has 5 saved parties, the save option should be replaced with a replace prompt.

Test ID	TC-10
Category	PvP – Invitation and Battle
Requirements Coverage	UC3 – Send a PvP invitation
Initial Condition	Two users are registered: player1 and player2. Both have at least one saved party. player1 is logged in.
Procedure	1. As player1 (Password: password), navigate to /pvp by pressing “PVP BATTLE” button on dashboard. 2. Enter player2 in the invitation field and click Send. 3. Log in as player2 (Password: password) in a separate browser session. 4. Navigate to /pvp. 5. Accept the invitation from player1. 6. Both players select a saved party.
Expected Outcome	Both players are redirected to the battle view. Battle proceeds normally. On conclusion, the winner's pvpWins counter increments by 1 and the loser's pvpLosses increments by 1 in the database. Both players are redirected to /pvp after the battle.
Notes	Inviting a username that does not exist, or that does not have a saved party should show an error.

	A user without a saved party should not be able to send or accept invitations.
--	--

USE CASE 4

Test ID	TC-11
Category	Battle System - Loss Handling
Requirements Coverage	UC3 – Single Battle
Initial Condition	User is in an active PvE battle. All player heroes are close to 0 HP.
Procedure	1. Allow all player heroes to be defeated (take damage until HP reaches 0 on all units). 2. On the battle result page, note current hero experience and party gold. 3. Click Continue. User enters inn if previously encountered.
Expected Outcome	Each hero loses 30% of experience gained in the current level (cannot go below level threshold). Party loses 10% of current gold. If the inn has been visited, player is redirected to the inn. Otherwise, campaign is abandoned and player is redirected to the dashboard.
Notes	Heroes cannot lose levels — experience floor is the start of the current level. Gold penalty is floored at 0.

Test ID	TC-12
Category	Inventory – Item Use
Requirements Coverage	UC2 – PvE campaign (inventory management)
Initial Condition	User is on the campaign page. Party inventory contains at least one food or drink item. At least one hero has less than full HP or mana.
Procedure	1. On the campaign page, open the inventory panel. 2. Select a hero with less than full mana. 3. Use a Water item on that hero.
Expected Outcome	Hero's mana increases by 10 (capped at max mana). Water is removed from inventory. Campaign page refreshes showing updated mana and updated inventory.

Notes	Items cannot be used during a battle — the use-item button should only be available on the campaign page. Using an Elixir on a dead hero should revive them and restore full HP and mana.
--------------	---